

第 17 讲：Long-running Harness：让 Coding Agent 可以暂停、恢复与回放

Coding Agent Notes (June 2026)

本讲在系列中的位置： #11 讲产品工作流，#13-#16 讲 context 与 memory；本讲讲一个任务跨越多轮、多人、多进程、多天时，harness 怎样保持连续性。

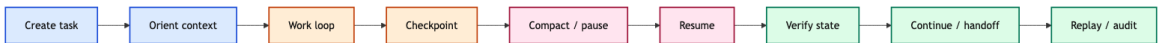
已知前提： 读者已经知道 harness loop、workspace、permission、context engineering、trajectory 和 memory。

本讲新增： durable session、event log、base state、workspace checkpoint、runtime persistence、resume gate、restore receipt、replay audit。

读完后应该能回答： 为什么恢复聊天记录不等于恢复任务？为什么 checkpoint 不能只保存文件？为什么 resume 之前必须验证状态？

1 核心图景

长期 coding agent 的难点，不是让模型一次读更多 token，而是让一个真实软件工程任务在暂停、压缩、崩溃、切换、恢复、转交之后仍然能继续推进。一个 long-running harness 要解决的问题是：当 agent 说“我接着做”，它到底接上了什么？



这条生命周期的核心不是“工作时间很长”，而是每一次停止和恢复都要保持状态一致。Create task 是任务入口；orient context 是加载当前 repo、规则、计划和证据；work loop 是常规行动；checkpoint 是保存可恢复状态；compact / pause 是上下文压缩或任务暂停；resume 是重新打开会话；verify state 是确认恢复后的世界和之前一致；handoff / replay 是给人类或另一个 agent 审计和接手。

Insight. Long-running harness 的本质是 durability contract：它承诺哪些状态被保存、哪些状态会丢、恢复后如何验证，以及如果状态不一致应该停下来而不是继续编。

2 长任务不是长上下文

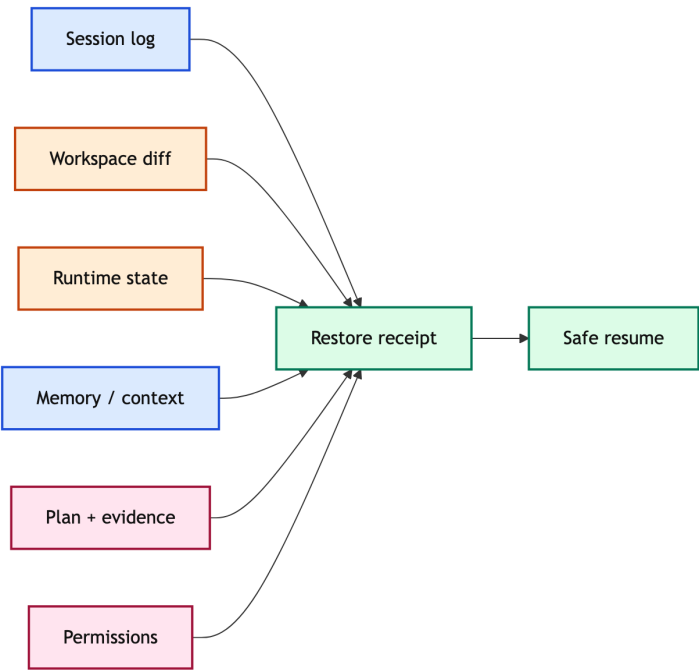
很多人会把 long-running agent 理解成“context window 更长”。这是不够的。长上下文只能保存更多文本，但软件工程任务的状态不只在文本里。文件被改过，测试跑过，进程可能还在，环境变量可能变了，用户批准过某个危险操作，agent 也可能已经排除了几个错误假设。这些都不是简单追加聊天记录就能恢复的。

误解	真正的问题	典型后果
只要 context 足够长	工作区和 runtime 也在变化	agent 记得讨论，但看错当前文件状态
resume 就是接着聊天	resume 需要状态重建和验证	继续执行过期计划，破坏已有 diff
checkpoint 只要保存文件	进程、环境、权限、证据也可能重要	测试或 server 状态无法复现
compaction 只是摘要	摘要会丢失细节和拒绝过的路径	agent 重复探索或忘记约束

Claude Code 文档里把 context window 描述为会话历史、文件内容、命令输出、项目指令、auto memory、skills 和系统指令的组合，并指出上下文会随着工作填满、需要压缩。这个事实给我们的启发是：context 是运行时资源，不是持久化系统。真正的 long-running harness 必须有独立于 context window 的状态存储和恢复协议。

3 六个状态平面

一个可恢复的 coding-agent session 至少有六个状态平面：session log、workspace diff、runtime state、memory/context、plan/evidence、permissions。它们必须被分开看，因为每一层的生命周期、恢复方式和失败模式都不同。



状态平面	保存什么	恢复时要验证什么
Session log	用户消息、agent 回复、工具调用、观察结果	日志是否完整，是否对应当前任务
Workspace diff	文件修改、git branch、untracked files	diff 是否仍然存在，是否被人改过
Runtime state	进程、server、环境变量、解释器变量	是否需要重启，是否仍然可信
Memory / context	项目规则、历史经验、压缩摘要	是否过期，是否被重新加载
Plan + evidence	当前计划、已跑测试、失败排除记录	下一步是否仍然成立
Permissions	用户批准、沙箱模式、危险操作边界	批准是否仍然适用

如果这六个平面没有对齐，agent 的行为会非常危险。它可能拿着旧日志里的计划，在新文件状态上继续改；也可能以为 server 还在运行，但实际进程已经丢失；或者记得用户曾批准过一次删除操作，却不知道那次批准只适用于旧路径。

4 状态错配的具体事故

把 long-running harness 做成“恢复聊天”最容易出现的事故，是状态错配。错配不一定立刻报错，反而经常以更隐蔽的方式出现：agent 继续工作，看起来很自信，但它基于的世界已经不是当前世界。

错配类型	例子	造成的损害
Log-workspace mismatch	日志说已修改 A，实际文件被人改回	agent 重复 patch 或覆盖人类修改
Plan-runtime mismatch	计划依赖 dev server，server 已停止	后续测试结论失效
Memory-workspace mismatch	记忆来自旧 branch，新 branch 架构已变	引入过期模式
Permission-state mismatch	旧批准对应旧路径，当前路径不同	权限被错误延续
Evidence-plan mismatch	测试在旧 diff 上通过，新 diff 未验证	reviewer 误信结论

这类事故的共同点是：它们不是模型“不会写代码”，而是 harness 没有告诉模型当前状态是否仍然成立。LLM 很擅长根据旧上下文生成连贯叙述，所以状态错配时它可能给出一个听起来合理的解释。问题在于，合理叙述不能替代状态验证。

Insight. 长任务里最危险的不是 agent 忘记，而是 agent 以为自己记得。Durable harness 要把“记忆”降级为候选证据，把“当前状态验证”提升为继续执行的前置条件。

因此，long-running harness 应该把“unknown”当成一等状态。很多系统只显示 running / paused / done，但长期任务需要更细：workspace unknown、runtime unknown、evidence stale、permission expired、memory stale。只要状态未知，继续行动就必须先做最小验证。

5 Event Log 与 Base State

OpenHands 的 persistence 文档给了一个很清楚的分层例子：conversation persistence 既有 base state，也有 events。Base state 保存 agent configuration、execution status、statistics、metadata 等核心状态；events 则保存 message history、tool calls、observations 和所有 conversation events。事件是追加的，base state 会随状态变化被覆盖更新。

持久化对象	类比	为什么分开
base state	当前仪表盘	快速知道会话是否 running、stuck、done
event files	完整轨迹	支持 replay、审计、转换回 LLM messages
conversation id	会话身份	防止把错误任务恢复到同一目录
tools / agent config	执行契约	工具不匹配时恢复可能不安全

这个设计对我们理解 long-running harness 很重要：日志和状态不是同一个东西。日志告诉你发生过什么，base state 告诉你现在处于什么执行状态。只有日志没有 base state，恢复时需要从头推断当前进度；只有 base state 没有日志，又无法审计 agent 为什么到了这个状态。

Insight. 可恢复 session 应该同时有 append-only 的事件历史和可快速读取的当前状态。前者服务 replay 和审计，后者服务 resume 和调度。

6 Workspace Checkpoint

Coding agent 最特殊的状态是 workspace。聊天模型可以恢复对话，但代码任务必须恢复文件状态。一个任务如果改了 12 个文件、生成了临时数据、删除了某个目录、跑过 migration，那么 resume 只恢复 conversation 是不够的。

Gemini CLI 的命令文档提供了一个具体例子：/resume 用于浏览和恢复历史 session；/restore 在启用 checkpointing 时，可以把项目文件恢复到某次 tool execution 之前的状态。这里的关键区别是：resume 面向对话连续性，restore 面向文件状态回滚。

操作	主要恢复对象	不解决什么
Resume session	会话历史、用户意图、之前工具结果	文件是否仍是旧状态
Restore files	工作区文件快照	agent 是否理解为什么回滚
Git checkpoint	commit / branch / diff 边界	未纳入 git 的 runtime 状态
Manual handoff note	人类可读进度说明	机器可回放的完整事件

因此，long-running harness 需要把 workspace checkpoint 当作一等对象。最小做法是保存 git diff、branch、HEAD、untracked file 列表和最后一次验证命令。更强的做法是将每次会影响文件系统的工具调用前后都关联一个 checkpoint，这样用户可以回到“某次 edit 之前”而不是只回到“某次聊天之前”。

7 Checkpoint 粒度策略

Checkpoint 不应该只有“开”或“关”。不同任务需要不同粒度。只改一个 README 时，全量 sandbox checkpoint 过重；启动数据库、迁移 schema、开后台 server 时，只保存 git diff 又不够。一个实用 harness 应该根据状态变化选择 checkpoint 类型。

状态变化	推荐 checkpoint	理由
只读搜索、查看文件	不保存或只记录 event	没有 recovery-relevant state
编辑源码文件	diff / file snapshot	恢复主要依赖文件状态
生成大量文件	filesystem snapshot + manifest	untracked 文件也要可恢复
启动服务或 REPL	runtime receipt 或 process checkpoint	文件不足以恢复执行上下文
数据库迁移 / 外部副作用	强制确认 + evidence snapshot	需要明确不可逆边界

Crab 的研究给了一个更系统的版本：用 OS-visible effects 判断本轮是否需要 filesystem、process 或 full checkpoint，并把 checkpoint 工作尽量放在 LLM 等待窗口中异步完成。我们在 notes pipeline 里不需要实现这个系统，但应该吸收它的原则：checkpoint 的粒度要由“未来恢复需要什么状态”决定，而不是由工具名字决定。

例如，同样是 Bash command，‘grep’只是读；‘pytest’可能产生缓存和日志；‘npm run dev’可能留下长进程；‘alembic upgrade head’可能改变数据库。只看工具名都是 Bash 无法区分这些语义。Harness 必须至少记录命令类型、cwd、文件变化、进程变化和验证结果，才能决定恢复点是否完整。

Checkpoint policy 的最小版本：

读操作只进 event log；文件写操作保存 diff；长进程写 runtime receipt；不可逆或外部副作用必须要求人类确认并写 evidence note；恢复前显示最近一次完整 checkpoint 的状态范围。

8 Runtime Persistence

除了文件，runtime 也可能是状态。Python 解释器里的变量、long-running dev server、数据库容器、后台测试、临时 socket、环境变量，都可能影响下一步行为。如果 agent 恢复后只看到了旧聊天记录，却没有同一个 runtime，它可能继续引用不存在的变量或假设一个已经停止的 server 还在运行。

Agents Learn Their Runtime 这篇论文把 persistent interpreter 和 stateless interpreter 当成训练/部署语义来研究。它报告了两个很有启发的现象：persistent-trained model 被放到 stateless runtime 时会

产生大量 missing-variable 错误；stateless-trained model 即使在 persistent runtime 里，也可能继续重复重建状态，产生约 3.5x 的 token 开销。这里的数字只属于那篇论文的设置，但结论很适合本讲：runtime persistence 是 harness contract，不是实现细节。

Runtime 契约	Agent 学到的行为	mismatch 后果
Persistent	变量和中间结果可以跨步复用	换成 stateless 后会引用不存在状态
Stateless	每步重新显式构造所需状态	换成 persistent 后仍然重复劳动
Mixed / unclear	agent 不知道状态在哪里	token 浪费、错误恢复、测试不稳定

对 coding agent 来说，这意味着 prompt、tool schema、训练轨迹和部署 runtime 必须一致。如果你的生产 harness 会保持 shell cwd、环境变量和 server 状态，训练数据最好也暴露这种连续性；如果生产 harness 每次命令都是独立进程，就不能让模型以为上一步定义的变量仍然存在。

这里还有一个常见设计选择：把状态放进 context，还是放进 runtime？放进 context 的好处是透明，模型能读到；坏处是占 token，而且会随 compaction 损失细节。放进 runtime 的好处是高效，真实执行环境能保留变量和进程；坏处是模型可能过度依赖不可见状态，一旦 runtime 重置就崩。

状态位置	优点	风险
Context text	可读、可审计、可压缩	token 成本高，摘要可能丢细节
Workspace file	可版本化，可被人类 review	需要命名和清理策略
Interpreter / process	高效，适合多步计算和 server	恢复难，mismatch 隐蔽
External memory	跨 session，可检索	过期、权限和相关性难管理

一个好的 harness 不会把所有状态都塞进一个地方。计划和证据应该写成短文本；源码变化应该进入 workspace diff；长日志应该归档；运行中 server 应该有 runtime receipt；长期经验应该进入 memory；不可恢复的外部副作用应该进入权限和审计记录。

9 Sandbox Checkpoint / Restore

更完整的长期任务会碰到 sandbox checkpoint / restore。Crab 论文把问题说得很尖锐：agent sandboxes 会积累 OS state，而 chat-only recovery 会漏掉 OS-side effects；全量每轮 checkpoint 又太贵。它提出 agent-OS semantic gap：agent 层看得到 tool calls，但不一定看得到真实 OS 影响；OS 层看得到文件和进程变化，但不理解这些变化对 agent 任务是否重要。

Sandbox 状态	为什么重要	只保存聊天会怎样
Filesystem	patch、生成文件、缓存、测试产物	恢复后文件和计划不一致
Process state	server、REPL、后台任务、数据库进程	agent 以为服务还在或变量还在
Environment	PATH、venv、env vars、cwd	命令行为和上次不一致
Checkpoint manifest	哪些状态组合成一个恢复点	部分恢复导致状态错配

Crab 的启发不是“每个 notes 项目都要实现 CRIU/ZFS”，而是 checkpoint 要按语义选择粒度。很多 SWE-style bug fix 只需要 filesystem checkpoint；terminal-heavy 或 server-heavy 任务可能需要 process state；多数 turn 可能根本不产生 recovery-relevant state。一个成熟 harness 应该能判断当前 turn 是否需要 checkpoint、需要哪种 checkpoint、以及恢复后是否需要 replay 某些逻辑进度。

10 Compaction、Memory 与 Resume

长任务一定会遇到 compaction。压缩能让 session 继续,但也会改变 agent 能看到的内容。Claude Code 文档指出,project-root CLAUDE.md 可以在 compaction 后重新注入,而 conversation-only instruction 可能丢失;auto memory 的 MEMORY.md 在启动时只加载前 200 行或 25KB,详细 topic files 需要按需读取。这说明 memory 不是无限上下文,而是有预算、有入口、有加载策略的状态平面。

上下文来源	生命周期	设计原则
Conversation history	会随 compaction 被摘要	不放不可丢的项目规则
CLAUDE.md / AGENTS.md	会话启动加载的项目规则	短、结构化、可验证
Auto memory	跨 session 的本地记忆	做索引,不堆长流水账
Hook context	按事件动态注入	快、短、可审计
Skills / subagents	按需加载或隔离上下文	避免主 session 膨胀

Hooks 是 long-running harness 的重要机制。Claude Code 的 SessionStart 可以区分 startup、resume、clear、compact; SessionStart hook 的输出可以进入模型上下文; CLAUDE_ENV_FILE 可以让 hook 写入后续 Bash 命令可见的环境变量。这些机制说明 resume 可以做成一个主动恢复流程:加载最近 issue、当前 branch、未提交 diff、上次测试、active plan,而不是只把旧聊天交给模型。

11 Pause Protocol

Resume 是否可靠,取决于 pause 时有没有留下足够状态。很多系统只在恢复时做补救,但最好的恢复发生在暂停之前。Pause protocol 应该要求 agent 在停止前收束状态:保存 diff,写当前计划,记录最后验证,列出未解决风险,标记下一步,以及说明哪些 runtime 状态不能恢复。

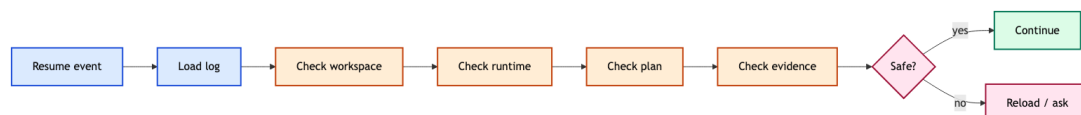
Pause 产物	内容	恢复时用途
Progress note	已完成、未完成、下一步	防止重复规划
Evidence note	最后一次测试、日志、截图、失败	判断结论是否过期
Diff manifest	修改文件、生成文件、删除文件	检查 workspace 是否被改动
Runtime note	server、容器、env、cwd、后台任务	判断是否需要重启
Risk note	不确定假设、拒绝路径、权限边界	防止恢复后盲目推进

这可以用 hook、stop rule 或普通 agent instruction 实现,但关键是产物要短而结构化。暂停摘要不应该是“我完成了大部分工作,接下来继续测试”这种空话,而应该让任何接手者能回答:现在能不能继续 edit? 需要先跑什么命令? 哪些文件是 intentional diff? 哪些操作需要重新批准?

Insight. 好的 pause note 是未来 resume 的输入,而不是过去工作的纪念品。它必须服务下一步行动。

12 Resume Gate

安全恢复需要一个 resume gate。它不是一个 UI 按钮,而是一套检查:读取日志、确认 workspace、确认 runtime、确认计划、确认证据、确认权限,然后决定是否继续。最重要的产物是 restore receipt:一段短而结构化的说明,告诉 agent 和人类“恢复了什么、没恢复什么、下一步是否安全”。



检查项	具体问题	失败时动作
Log	事件是否连续? 最后一步是什么?	不继续, 先重建摘要
Workspace	HEAD、diff、untracked 是否匹配?	显示差异, 要求确认
Runtime	server、venv、env、cwd 是否可信?	重启或标记不可恢复
Plan	下一步是否仍然适用?	更新计划, 删除过期假设
Evidence	最近测试和失败证据是否有效?	重新跑最小验证
Permission	旧批准是否仍然适用?	重新请求敏感操作批准

Resume gate 的关键是默认保守。状态不确定时, 不应该让 agent 用自然语言编一个“看起来合理”的连续性故事。正确做法是先展示 receipt: 哪些状态恢复成功, 哪些状态缺失, 哪些需要重新验证。只有 receipt 通过, agent 才能继续 edit / test / commit。

13 Restore Receipt 格式

Restore receipt 是 long-running harness 的核心交互对象。它不是给论文看的日志, 而是给 agent、人类 reviewer 和下一个 session 共同看的短状态报告。它应该足够短, 能在 resume 时进入 context; 又要足够具体, 能阻止错误继续。

Receipt 字段	示例内容	作用
Task id / branch	issue-4211, feat/auth-refactor	防止恢复错任务
Workspace state	HEAD、modified files、untracked files	检查文件状态
Last evidence	pytest auth tests passed; e2e not run	知道哪些结论有效
Runtime state	server stopped; venv needs activation	避免假设进程仍在
Open risks	migration rollback untested	指导下一步验证
Safe-to-resume	yes / no / unknown + reason	给继续行动一个闸门

Receipt 的最后一行最重要: safe-to-resume 不能默认 yes。比如 workspace diff 和 checkpoint 不一致时, safe-to-resume 应该是 unknown; runtime 依赖的服务已经停止时, 应该要求先重启或重新跑测试; 旧 permission 超过当前状态范围时, 应该重新请求批准。

这类 receipt 还可以帮助人类审查 agent。一个 PR reviewer 不需要读完所有聊天, 只需要先看 receipt: agent 是否知道自己改了什么? 是否知道哪些测试没跑? 是否知道风险? 如果 receipt 说不清, 说明这个 session 的 long-running 状态管理还不够成熟。

14 Replay 与 Audit

Replay 不是简单重放聊天记录。一个可审计 replay 至少要能回答四个问题: agent 看到了什么 observation, 做了什么 action, 为什么认为这个 action 合理, 结果如何改变 workspace/runtime。OpenHands 的事件文件可以转回 LLM-ready messages, 这是 replay 的一部分; 但软件工程 replay 还需要文件状态和验证证据。

Replay 层次	能回答的问题	不足
Chat replay	谁说了什么, 模型如何回答	不保证文件状态一致
Tool replay	哪些命令/编辑被执行	不一定能重现外部环境
State replay	每步前后 workspace/runtime 状态	成本更高, 需要 checkpoint
Evidence replay	哪些测试、截图、日志支持结论	需要结构化保存证据

对团队协作来说, replay 的价值不是复现每一个 token, 而是让 reviewer 能判断 agent 的结论是否可

信。比如：它为什么修改这个文件？它是否看过调用方？它有没有跑回归测试？它是否在 compaction 后忘记了一个限制？这些问题如果只能靠翻长聊天来回答，long-running harness 就没有真正成熟。

Replay 还要区分 deterministic replay 和 semantic replay。Deterministic replay 想完全重放每一步，这在涉及外部服务、时间、网络、随机数、依赖版本时很难。Semantic replay 则只要求复原关键决策链：输入是什么，证据是什么，动作是什么，验证是什么，结论是什么。对多数 coding agent 产品，semantic replay 更实际。

Replay 目标	需要保存	成本
Debug model behavior	prompts、tool calls、observations	中等，主要是日志体积
Reproduce workspace	file snapshots、diff、dependency versions	较高，需要存储策略
Reproduce runtime	process/container snapshot、env、ports	高，且平台相关
Audit conclusion	evidence receipt、test output、risk note	较低，但需要结构化

这也是为什么 event log 和 evidence note 都重要。Event log 保留细节，但太长；evidence note 很短，但可能遗漏。长期 harness 应该让二者互相索引：receipt 指向关键 event 和 checkpoint，event log 可以追溯 receipt 的来源。

15 Handoff: 从一个 Agent 到另一个 Agent

长任务常常需要 handoff：从主 agent 交给 subagent，从实现者交给 reviewer，从本地 CLI 交给云端任务，或者从今天的自己交给明天的自己。Handoff 成功的条件不是“写一段总结”，而是交付一个可验证任务包。

Handoff artifact	内容	好坏判断
Task state	目标、范围、已完成、未完成	下一位 agent 能立刻定位工作
Diff state	改了哪些文件，为什么改	能区分 intentional 和 accidental
Evidence state	跑过哪些测试，结果如何	reviewer 能复核结论
Risk state	已知风险、拒绝路径、待确认事项	不重复踩坑，不隐藏不确定性
Resume receipt	哪些状态已恢复，哪些未恢复	接手者知道能否继续执行

这个设计和 #12 的多 agent 直接相连。多 agent 如果没有 durable state，就会变成多份不一致的聊天摘要；每个 agent 都以为自己理解任务，但实际看到的 workspace、计划和证据不同。Handoff 不是多写几句自然语言，而是把状态面打包成可检查的交接对象。

16 产品设计启发

一个面向团队的 long-running harness，应该把状态连续性做成用户可见的产品能力。用户不应该只看到“Resume”按钮，还应该看到当前 branch、diff、last checkpoint、last tests、pending permissions、active plan、open risks。否则，resume 只是把风险隐藏起来。

产品能力	用户看到什么	系统意义
Task timeline	创建、暂停、恢复、checkpoint、handoff	状态变化可追踪
Checkpoint list	每个恢复点关联 diff 和 tool call	可回滚到工程状态
Resume receipt	当前恢复是否安全	防止假连续性
Evidence panel	最近测试、截图、日志、失败原因	支撑 review 和 PR
Permission ledger	哪些批准仍有效，哪些过期	敏感动作不跨状态滥用

这也解释了为什么 long-running harness 和安全不是两件事。状态越长，权限越容易被误用；任务越久，证据越容易过期；resume 越频繁，越需要清楚区分“我记得”与“我验证过”。一个真正可靠的 coding agent 产品，会把“继续工作”变成有条件动作，而不是无条件动作。

17 内部实验怎么做

团队不需要先实现完整 checkpoint/restore 系统，才能评估 long-running harness。可以先做一个最小实验：选择 20 个需要 30 分钟以上、多轮测试、多次编辑的真实任务；让 agent 在中途强制 pause / compact / resume；比较有无 restore receipt、有无 workspace snapshot、有无 resume gate 的表现差异。

实验设置	记录指标	目的
Chat-only resume	resolved、重复探索次数、错误假设	测 baseline 风险
Workspace snapshot	diff mismatch、restore 成功率	测文件连续性
Resume gate	继续前阻断次数、误继续次数	测安全恢复
Evidence receipt	reviewer 复核时间、测试遗漏	测协作价值
Runtime reset vs persist	命令失败、重启次数、token 成本	测 runtime 契约

每个失败样例都应该归因到一个状态平面：log 缺失、workspace 错配、runtime 丢失、memory 过期、plan 过期、evidence 不足、permission 不清。这样团队才能知道下一步该做日志、checkpoint、hook、memory、UI，还是权限系统。

最小验收标准：

恢复后，agent 必须能说清楚当前 HEAD、未提交 diff、上次执行的验证、当前计划、未解决风险、是否有状态缺失。任何一项 unknown，都应该先验证或询问，而不是继续修改代码。

18 实现路线

如果要从零开始实现一个 long-running harness，不要直接上 full sandbox checkpoint。更稳的路线是分四层推进。第一层是 event log：所有用户消息、工具调用和观察都可追踪。第二层是 workspace manifest：每次写文件后记录 HEAD、diff summary、untracked files。第三层是 restore receipt：每次 pause / resume 生成短状态报告。第四层才是 runtime checkpoint：对 server-heavy、terminal-heavy、RL rollout-heavy 任务保存进程或容器状态。

阶段	交付物	通过标准
L1 Event log	JSONL trajectory + task id	能回答发生过什么
L2 Workspace manifest	HEAD、diff、untracked、last test	能判断文件状态是否一致
L3 Restore receipt	safe-to-resume gate	能阻止未知状态继续执行
L4 Runtime checkpoint	env/process/container snapshot	能恢复 server-heavy task

这条路线的好处是每一层都有独立价值。即使没有 runtime checkpoint，workspace manifest 也能避免覆盖人类修改；即使没有完整 replay，restore receipt 也能让 agent 不再盲目继续。不要把 long-running harness 设计成一个全有或全无的大系统。

19 和后续主题的关系

#18 会讲 skills，skills 是可复用的 procedural memory；但如果没有 durable session，skills 也可能在错误状态下被调用。#19 会讲 trajectory integrity，工具格式和终止信号如果不完整，event log 就不

能支持 replay。#20 会讲 tool design，工具 schema 需要暴露足够状态，才能让 checkpoint 和 receipt 有数据来源。

后续主题	与 long-running harness 的连接	本讲留下的接口
Agent Skills	skills 需要知道何时可调用	状态前置条件和适用范围
Trajectory Integrity	replay 依赖完整工具轨迹	event log 和 stop condition
Tool Design	工具要返回可恢复状态证据	observation schema 和 checkpoint hint
Sandbox / Permission	长任务放大副作用风险	permission ledger 和 restore gate
Git / PR Workflow	最终交付要能被 review	evidence panel 和 handoff note

因此，本讲不是一个孤立的“会话管理”主题，而是后面工具、权限、轨迹、交付的底座。只要 agent 可以跨时间工作，它就必须跨时间保存、解释和验证状态。

20 局限与开放问题

Long-running harness 会引入成本。事件日志、workspace snapshot、runtime checkpoint、memory 文件、hook 输出都可能占用存储和上下文。更严重的是隐私和安全：日志可能包含密钥、用户数据、内部路径；checkpoint 可能保存敏感文件；hook 可能以用户权限执行命令。Claude Code hooks 文档也提醒，command hooks 以系统用户权限运行，因此必须审计和最小化。

还有一个开放问题是 checkpoint 粒度。每一步都全量保存最安全，但成本太高；只保存聊天最便宜，但恢复不正确。Crab 的思路是根据 OS-visible effects 判断是否需要文件、进程或完整 checkpoint。对普通 coding agent 产品来说，也可以采用简化版：无文件变化时不保存；文件变化时保存 diff；启动 server 或数据库时记录 runtime receipt；涉及危险操作时强制用户确认和 evidence snapshot。

开放问题	风险	可能方向
Checkpoint 成本	保存太多拖慢任务	按状态变化选择粒度
Memory 过期	旧规则误导新任务	加更新时间和适用范围
隐私泄漏	日志和快照含敏感数据	redact、retention、access control
权限延续	旧批准被错误复用	权限绑定状态和时间
Replay 正确性	外部服务不可重现	保存 receipt，而不是假装全复现

21 最终 Takeaway

Long-running harness 让 coding agent 从“会话工具”变成“可持续工程系统”。它不是把聊天变长，而是把任务状态拆成可以保存、恢复、验证、回放和转交的对象。

本讲最重要的结论是：resume 必须经过验证。只有当 session log、workspace、runtime、memory、plan/evidence 和 permission 都被正确对齐，agent 才能安全地继续工作。否则，所谓 long-running 只是一个更长、更难审计的错误轨迹。

关键词	在本讲中的含义
Durable session	可以跨暂停、压缩、恢复、转交保持状态一致的会话。
Restore receipt	恢复后说明哪些状态已恢复、哪些缺失、是否安全继续的短报告。
Event log	追加式轨迹记录，用于 replay 和 audit。
Base state	当前执行状态快照，用于快速 resume 和调度。
Workspace checkpoint	文件状态恢复点，通常要关联 tool call 或 diff。
Runtime persistence	解释器、进程、环境等跨步状态的执行语义。

参考资料

- [1] [OpenHands Persistence](#). conversation persistence, base state, event files, and event reuse
- [2] [OpenHands Conversation API](#). conversation IDs, persistent event log, and restore constraints
- [3] [Claude Code: How It Works](#). session history, resume, context window, compaction, and subagent isolation
- [4] [Claude Code Memory](#). CLAUDE.md, auto memory, MEMORY.md startup budget, and compaction behavior
- [5] [Claude Code Hooks](#). SessionStart, compaction hooks, additional context, and environment persistence
- [6] [Gemini CLI Commands](#). resume, manual checkpoints, and restore command behavior
- [7] [Agents Learn Their Runtime](#). persistent vs stateless runtime semantics and train-runtime mismatch
- [8] [Crab](#). semantics-aware checkpoint/restore for agent sandboxes